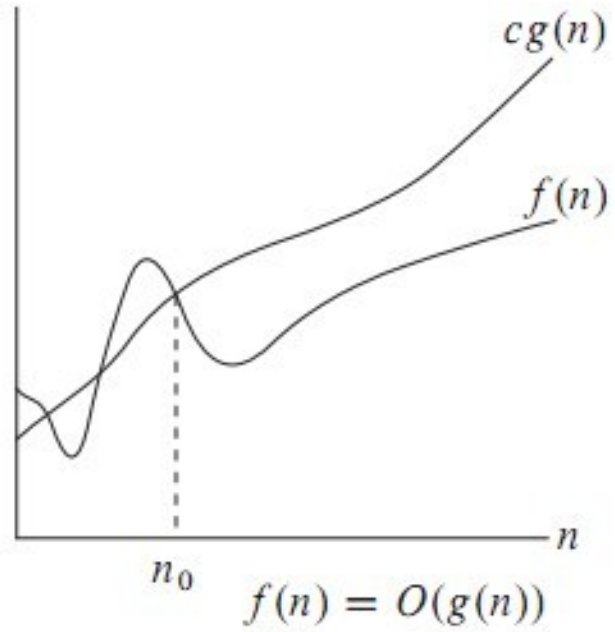
Big Oh (*O*)

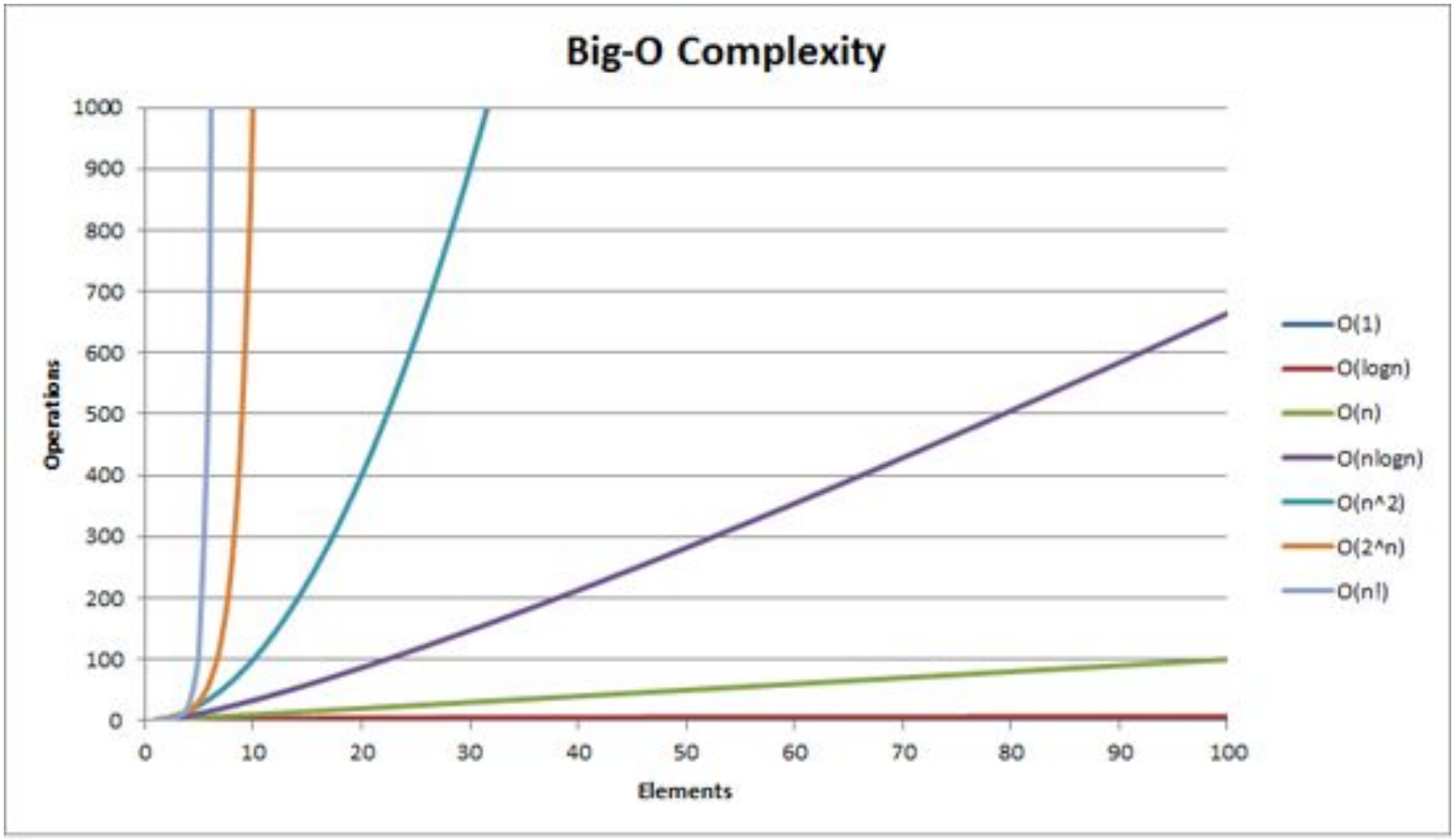
f(n)= O(g(n)) *iff* there exist positive constants c and n0 such that f(n) ≤ cg(n) for all n ≥ n0

O-notation to give an upper bound on a function



How bad is exponential complexity

- Fibonacci example – the recursive fib cannot even compute fib(50)



The class P

- The class P consists of those problems that are solvable in polynomial time.
- More specifically, they are problems that can be solved in time $O(n^k)$ for some constant k, where n is the size of the input to the problem
- The key is that n is the **size of input**

NP

- **NP is not the same as non-polynomial complexity/running time. NP does not stand for not polynomial.**
- **NP = Non-Deterministic polynomial time**
- NP means verifiable in polynomial time
- Verifiable?
  - If we are somehow given a ‘certificate’ of a solution we can verify the legitimacy in polynomial time

What happened to automata?

- Problem is in NP iff it is decidable by some non deterministic Turing machine in polynomial time.
- Remember that the model we have used so far is a deterministic Turing machine
- It is provable that a Non Deterministic Turing Machine is equivalent to a Deterministic Turing Machine
- Remember NFA to DFA conversion?
  - Given an NFA with n states how many states does the equivalent DFA have?
  - Worst case …. 2^n

- The deterministic version of a poly time non deterministic Turing machine will run in exponential time (worst case)

NP problems

- Graph theory has these fascinating(annoying?) pairs of problems
  - Shortest path algorithms?
  - Longest path is NP complete (we'll define NP complete later)
  - Eulerian tours (visit every vertex but cover every edge only once, even degree etc). Solvable in polynomial time!
  - Hamiltonian tours (visit every vertex, no vertices can be repeated). NP complete

Hamiltonian cycles

- Determining whether a directed graph has a Hamiltonian cycle does not have a polynomial time algorithm (yet!)
- However if someone was to give you a sequence of vertices, determining whether or not that sequence forms a Hamiltonian cycle can be done in polynomial time
- Therefore Hamiltonian cycles are in NP

SAT

- A boolean formula is ***satisfiable*** if there exists some assignment of the values 0 and 1 to its variables that causes it to evaluate to 1.
- CNF – Conjunctive Normal Form. ANDing of clauses of ORs

$$(x_0 \vee x_2) \wedge (\neg x_0 \vee x_1) \wedge (x_0 \vee x_1 \vee \neg x_2)$$

2-CNF SAT

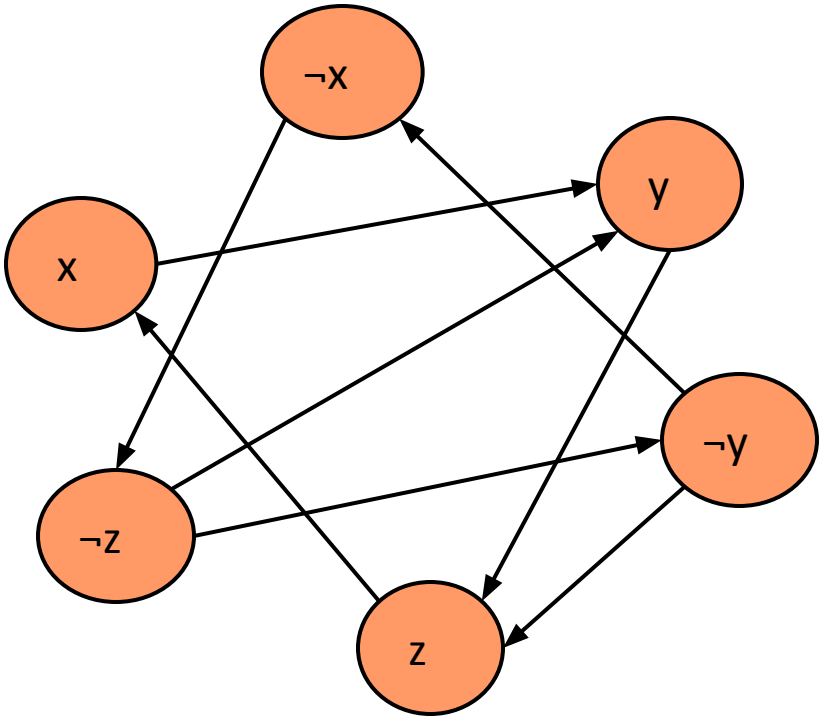
- Each OR operation has two arguments that are either variables or negation of variables
- The problem in 2 CNF SAT is to find true/false(0 or 1) assignments to the variables in order to make the entire formula true.

$$(\neg x \vee y) \wedge (\neg y \vee z) \wedge (x \vee \neg z) \wedge (z \vee y)$$

- Any of the OR clauses can be converted to implication clauses

2-SAT is in P

- Create the implication graph



Satisfiability via path finding

- If there is a path from x to $\neg x$
- And if there is a path from $\neg x$ to x
- Then FAIL!
- How to find paths in graphs?
  - DFS/BFS and modifications thereof

3 CNF SAT (3 SAT)

- Not so easy anymore.
- Implication graph cannot be constructed
- No known polytime algorithm
- Is it NP?
  - If someone gives you a solution how long does it take to verify it?
  - Make one pass through the formula and check
- This is an NP problem

P is a subset of NP

- Since it takes polynomial time to run the program, just run the program and get a solution
- But is NP a subset of P?
- No one knows if P = NP or not
- Solve for a million dollars!
  - http://www.claymath.org/millennium-problems
  - The Poincare conjecture is solved today

What is not in NP?

- Undecidable problems
  - Given a polynomial with integer coefficients, does it have integer roots
  - Impossible to check for all the integers
  - Even a non-deterministic TM has to have a finite number of states!
- Tautology
  - A boolean formula that is true for all possible assignments
  - Here just one ‘verifier’ will not work. You have to try all possible values

Amusing analogy

- Students believe that every problem assigned to them is NP-complete in difficulty level, as they have to find the solutions.
- Teaching Assistants, on the other hand, find that their job is only as hard as NP, as they only have to verify the student's answers.
- When some students confound the TAs, even verification becomes hard

Reducibility

- a problem Q can be reduced to another problem Q' if any instance of Q can be "easily rephrased" as an instance of Q', the solution to which provides a solution to the instance of Q
- Is a linear equation reducible to a quadratic equation?
  - Sure! Let coefficient of the square term be 0

NP - hard

- What are the hardest problems in NP?

$$L_1 \leq_p L_2$$

- That notation means that L_1 is reducible in polynomial time to L_2 .
- The less than symbol basically means that the time taken to solve L_1 is no worse that a polynomial factor away from the time taken to solve L_2.

NP-hard

- A problem (a language) is said to NP-hard if every problem in NP can be poly time reduced to it.

$$L' \leq_p L \text{ for every } L' \in NP$$

NP Complete problems/languages

- Need to be in NP
- Need to be in NP-Hard

If both are satisfied then it is an NP complete problem

Reducibility is a transitive relation.

If we know a single problem in NP-Complete that helps when we are asked to prove some other problem is NP-Complete

Assume problem P is NP Complete
All NP problems are reducible to this problem
Now given a different problem P'
If we show P reducible to P'
Then by transitivity all NP problems are reducible to P'

What is in NP-Complete

- For this course, we will axiomatically state that the following problems are NP-Complete
- SAT – Given any boolean formula, is there some assignment of values to the variables so that the formula has a true value
- 3-CNF SAT
- Actually any boolean formula can be reduced to 3-CNF form

Other NP complete problems

- Subset sum
- Given a set of positive integers and some target $t > 0$,

  do we have a subset that sums up to that target set
- Why is the naïve algorithm going to be bad?

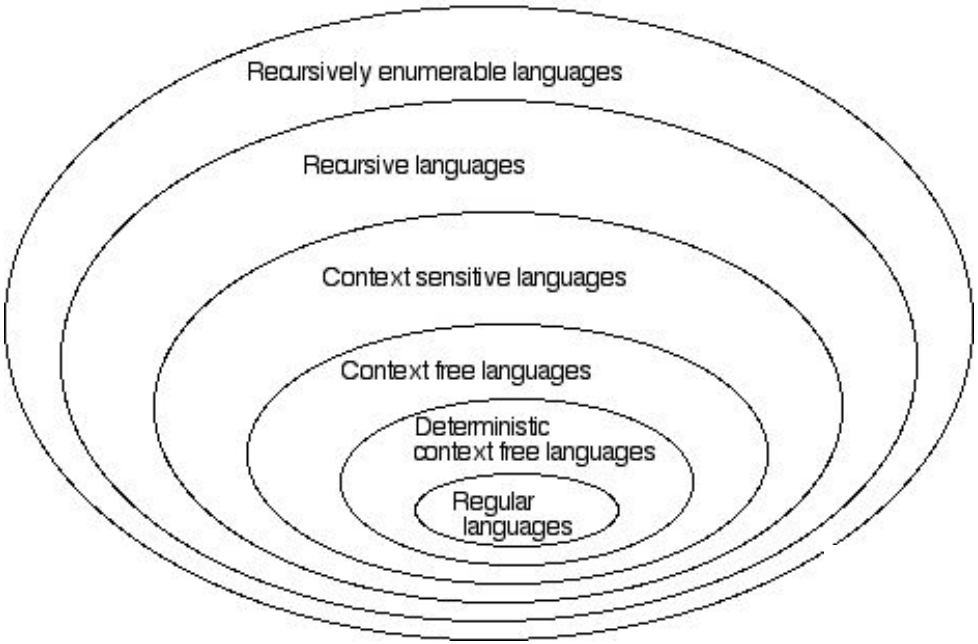
Church-Turing Thesis

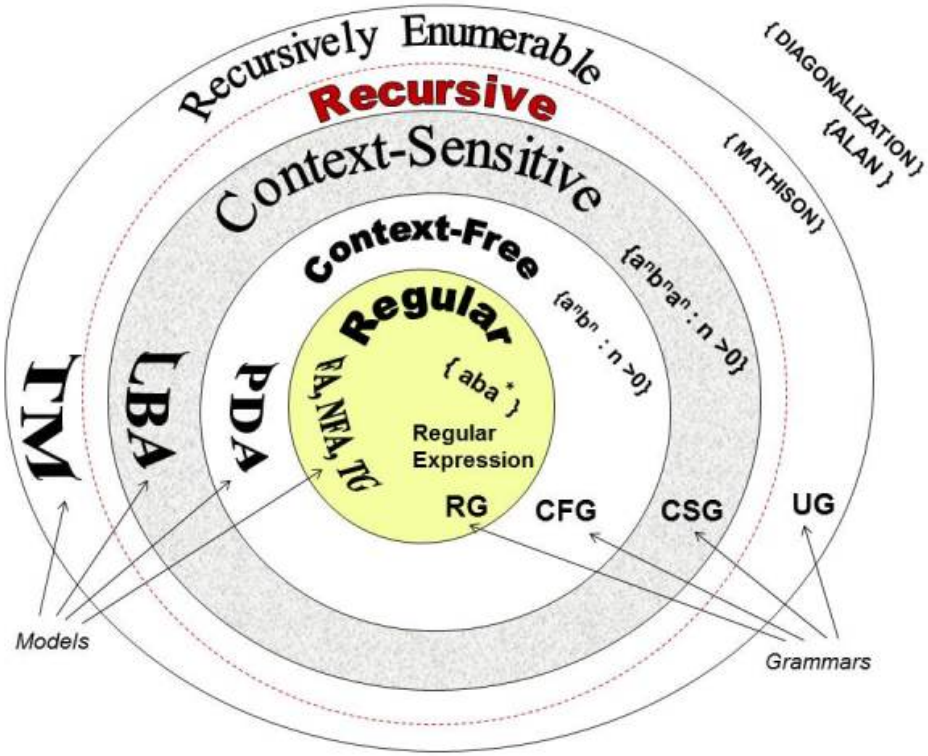
- The Church-Turing thesis (formerly commonly known simply as Church's thesis) says that any real-world computation can be translated into an equivalent computation involving a Turing machine.
- In Church's original formulation (Church 1935, 1936), the thesis says that real-world calculation can be done using the lambda calculus, which is equivalent to using general recursive functions.
- The Church-Turing thesis encompasses more kinds of computations than those originally envisioned, such as those involving cellular automata, combinators, register machines, and substitution systems.
- It also applies to other kinds of computations found in theoretical computer science such as quantum computing and probabilistic computing.

Church-Turing Thesis

- There are conflicting points of view about the Church-Turing thesis. One says that it can be proven, and the other says that it serves as a definition for computation.
- There has never been a proof, but the evidence for its validity comes from the fact that every realistic model of computation, yet discovered, has been shown to be equivalent.
- If there were a device which could answer questions beyond those that a Turing machine can answer, then it would be called an oracle.

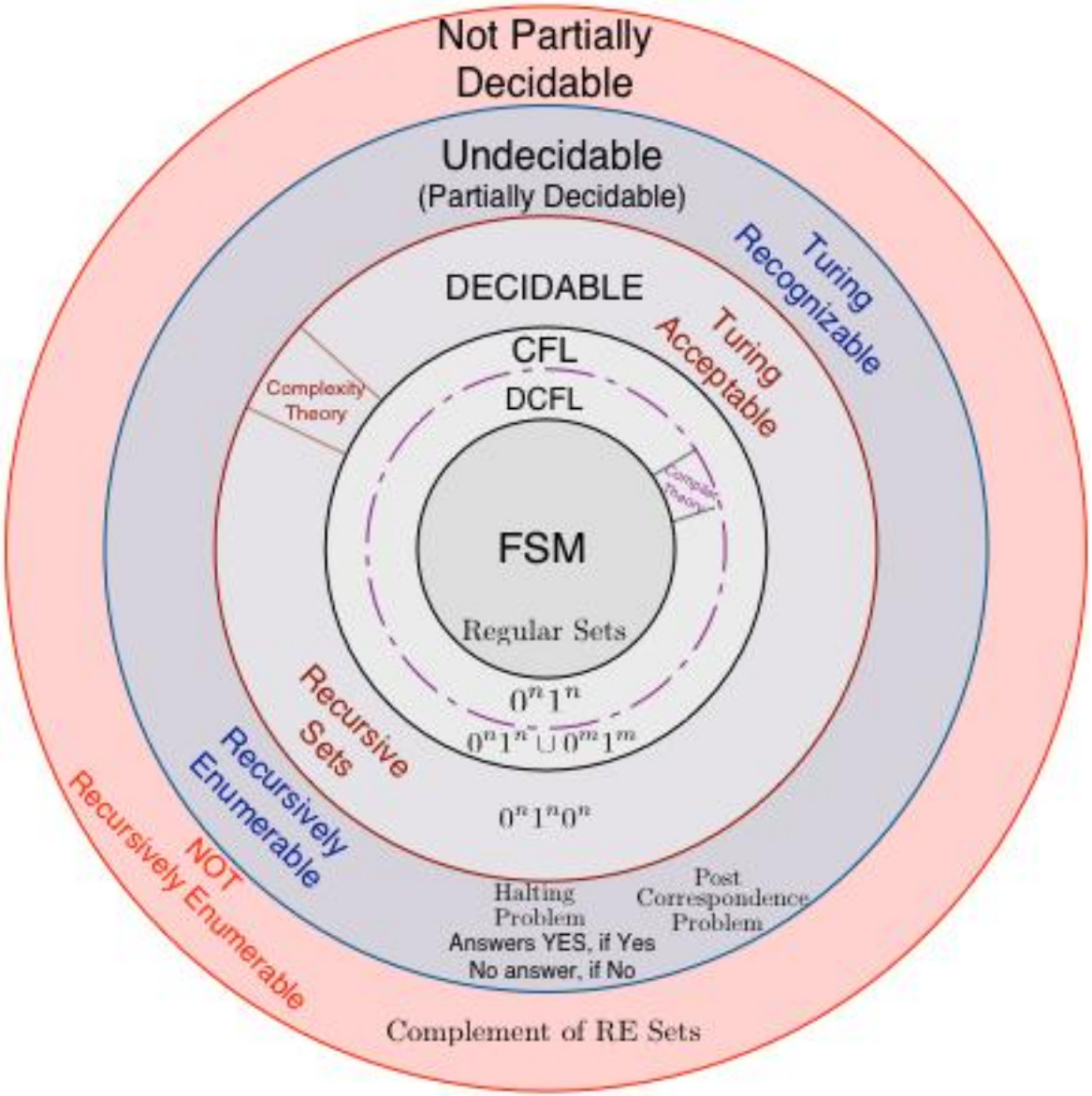
Elements of the Chomsky Hierarchy





Recursive language

- In mathematics, logic and computer science, a formal language (a set of finite sequences of symbols taken from a fixed alphabet) is called recursive if it is a recursive subset of the set of all possible finite sequences over the alphabet of the language.
- Equivalently, a formal language is recursive if there exists a total Turing machine (a Turing machine that halts for every given input) that, when given a finite sequence of symbols as input, accepts it if it belongs to the language and rejects it otherwise.
- **Recursive languages are also called decidable**.
- The concept of decidability may be extended to other models of computation.
- For example one may speak of languages decidable on a non-deterministic Turing machine.
- Therefore, whenever an ambiguity is possible, the synonym used for "recursive language" is Turing-decidable language, rather than simply decidable.


Not Partially Decidable
Undecidable
(Partially Decidable)
Turing Recognizable
DECIDABLE
Turing Acceptable
CFL
DCFL
Complexity Theory
Compiler Theory
FSM
Regular Sets
0^n1^n
$0^n1^n \cup 0^m1^m$
$0^n1^n0^n$
Recursive Sets
Recursively Enumerable
NOT Recursively Enumerable
Halting Problem
Answers YES, if Yes
No answer, if No
Post Correspondence Problem
Complement of RE Sets

Recursive Enumerable (RE) or Type -0 Language

- RE languages or type-0 languages are generated by type-0 grammars.
- An RE language can be accepted or recognized by Turing machine which means it will enter into final state for the strings of language and may or may not enter into rejecting state for the strings which are not part of the language.
- It means TM can loop forever for the strings which are not a part of the language.
- RE languages are also called as Turing recognizable languages.